

Q33. Application: Social Media Feed Ranking

A social media platform ranks posts using: $T(n) = 4T(n/2) + n^2$. Apply the Master Theorem to compute the time complexity. Clearly specify parameters and case selection. Analyze scalability issues as the number of posts increases and interpret efficiency.

Experiment No.: Q33

Application of Master Theorem in Social Media Feed Ranking System

Recurrence Relation: $T(n) = 4T(n/2) + n^2$

Submitted By

Name : T . KODANDA RAM

Register No : 192524162

Department : AI&DS

Course : CSA0619-Design and Analysis of Algorithms

Faculty : Dr. F. Mary Harin Fernandez

Date : 17/07/2026

AIM

To develop a **Social Media Feed Ranking System** using the **Divide and Conquer strategy** and analyze the recurrence relation $T(n) = 4T(n/2) + n^2$ using the **Master Theorem**, thereby determining its scalability, efficiency, and computational complexity.

OBJECTIVES

The objectives of this experiment are:

- To understand the concept of Divide and Conquer algorithms.
- To study recurrence relations used in recursive algorithms.
- To apply the Master Theorem for solving recurrence relations.
- To identify the values of **a**, **b**, and **f(n)**.
- To determine the appropriate Master Theorem case.
- To analyze the time complexity of the recurrence.
- To understand scalability and efficiency in recursive algorithms.
- To simulate a Social Media Feed Ranking System.

- To implement the algorithm in Python.
- To evaluate the algorithm using different input sizes.

PROBLEM STATEMENT

Modern social media platforms such as Facebook, Instagram, X (Twitter), and LinkedIn receive millions of posts every minute. Showing posts in chronological order often results in users missing the most relevant and engaging content.

To improve user experience, social media platforms rank posts based on multiple engagement metrics such as:

- Number of Likes
- Number of Comments
- Number of Shares
- Number of Saves
- Number of Views

A recursive feed-ranking algorithm divides the feed into smaller groups, processes them independently, and combines the ranking results.

The recurrence relation governing the algorithm is

$$T(n)=4T(n/2)+n^2$$

The objective is to analyze this recurrence using the Master Theorem, determine its computational complexity, study its scalability, and interpret its efficiency.

INTRODUCTION

Social media has become one of the largest sources of digital information. Every second, users upload thousands of photographs, videos, stories, and posts. Since users cannot view every available post, social media companies use intelligent ranking algorithms to determine which posts should appear first.

Feed ranking algorithms aim to maximize user engagement by displaying content that is most relevant to each user. These algorithms consider multiple engagement factors, including likes, comments, shares, saves, watch time, and user interactions.

As the number of posts increases rapidly, ranking algorithms must process very large datasets efficiently. Recursive algorithms based on the Divide and Conquer paradigm provide an effective solution by breaking the problem into smaller subproblems.

Instead of processing all posts simultaneously, the algorithm repeatedly divides the feed into smaller groups, ranks each group independently, and merges the results. This recursive process continues until the smallest possible subproblem is reached.

The efficiency of recursive algorithms is represented using **recurrence relations**. Solving these recurrence relations allows us to predict how the algorithm performs as the dataset size increases.

For this experiment, the recurrence relation is:

$$T(n)=4T(n/2)+n^2$$

The Master Theorem provides a systematic method to determine the asymptotic running time without expanding the recurrence manually. By identifying the values of **a**, **b**, and **f(n)**, we can determine the appropriate case of the theorem and derive the final time complexity.

This experiment demonstrates how mathematical analysis and algorithm design work together to evaluate the efficiency of recursive algorithms used in real-world applications such as social media feed ranking.

THEORY

Social Media Feed Ranking System

A Social Media Feed Ranking System is an algorithm that determines the order in which posts are displayed to users. Instead of displaying posts in chronological order, the system ranks them according to their importance and relevance.

Each post is assigned a ranking score based on various engagement metrics.

Typical ranking parameters include:

- Number of Likes
- Number of Comments
- Number of Shares
- Number of Saves
- Number of Views
- User Interaction History
- Trending Score
- Content Quality
- User Preferences

After calculating the ranking score, posts are sorted in descending order so that the highest-ranked posts appear first in the user's feed.

In this experiment, ranking is simulated by generating sample posts with random engagement values and computing an overall score for each post.

The algorithm recursively divides the dataset into smaller groups and processes them independently, making it suitable for recurrence analysis using the Master Theorem.

WHY SOCIAL MEDIA FEED RANKING IS IMPORTANT

Social media platforms receive billions of interactions every day. Without an efficient ranking system, users would be overwhelmed with irrelevant content.

An effective feed-ranking system offers several benefits:

- Displays the most relevant content first.
- Improves user engagement.
- Reduces information overload.
- Personalizes the user experience.
- Increases time spent on the platform.
- Improves advertisement targeting.
- Enhances content discovery.
- Supports scalable processing of massive datasets.

Because the ranking process involves handling very large amounts of data, efficient algorithm analysis becomes essential.

THEORETICAL BACKGROUND AND MATHEMATICAL ANALYSIS

DIVIDE AND CONQUER TECHNIQUE

Definition

Divide and Conquer is one of the most important algorithm design paradigms in computer science. It solves a large problem by dividing it into smaller subproblems, solving each subproblem recursively, and combining their solutions to obtain the final result.

Instead of processing a large dataset at once, the algorithm repeatedly divides the input until the problem becomes small enough to solve directly.

Many efficient algorithms are based on the Divide and Conquer strategy because it simplifies complex problems and improves computational efficiency.

Examples include:

- Merge Sort
- Quick Sort
- Binary Search
- Strassen Matrix Multiplication
- Closest Pair of Points
- Karatsuba Multiplication

In this experiment, the Social Media Feed Ranking algorithm follows the Divide and Conquer strategy by recursively splitting the collection of posts into smaller groups, processing each group independently, and combining the ranking results.

Working Principle of Divide and Conquer

The Divide and Conquer approach consists of three major steps.

Step 1: Divide

The original dataset containing n posts is divided into smaller equal-sized datasets.

Example:

128 Posts

↓

64 Posts + 64 Posts

↓

32 + 32 + 32 + 32

↓

...

Step 2: Conquer

Each smaller dataset is processed recursively.

If only one post remains, the recursion stops because no further division is possible.

Step 3: Combine

The ranking results obtained from the smaller datasets are merged to produce the final ranked feed.

This recursive process continues until the complete ranked feed is generated.

SOCIAL MEDIA FEED RANKING USING DIVIDE AND CONQUER

The feed-ranking algorithm processes a large number of posts using recursion.

Initially,

n Posts

are divided into

4 Recursive Subproblems

Each recursive call processes

$n/2$ Posts

After processing,

all ranked subfeeds are combined to obtain the final ranking.

Since four recursive calls are made, the recurrence becomes

$$T(n)=4T(n/2)+n^2$$

where

- **$4T(n/2)$** represents four recursive calls.
- **n^2** represents the ranking and merging operations performed at each recursive level.

RECURRENCE RELATION

A recurrence relation is a mathematical equation that expresses the running time of a recursive algorithm in terms of the running time of smaller subproblems.

For this experiment,

$$T(n)=4T(n/2)+n^2$$

Where,

Symbol	Meaning
$T(n)$	Time required to process n posts
4	Number of recursive calls
$n/2$	Size of each subproblem
n^2	Feed ranking work at every recursive level

The recurrence relation describes how the running time grows as the number of posts increases.

MATHEMATICAL BACKGROUND

To solve recursive algorithms efficiently, expanding the recurrence manually is often impractical for large values of **n**.

Instead, computer scientists use the **Master Theorem**, which provides a direct method for determining the asymptotic running time of Divide and Conquer algorithms.

The general form of the Master Theorem is

$$T(n)=aT(n/b)+f(n)$$

Where,

Parameter	Meaning
a	Number of recursive calls
b	Factor by which problem size is reduced
f(n)	Additional work performed outside recursion

The theorem compares

$f(n)$

with

$n^{\log_b a}$

to determine the correct complexity.

MASTER THEOREM

The Master Theorem is one of the most widely used techniques for analyzing Divide and Conquer algorithms.

Its general recurrence is

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where,

- $a \geq 1$
- $b > 1$
- $f(n)$ represents the non-recursive work.

The theorem consists of three cases.

Case 1

If

$$f(n) = O(n^{\log_b a - \varepsilon})$$

for some

$$\varepsilon > 0$$

then

$$T(n) = \Theta(n^{\log_b a})$$

Case 2

If

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Case 3

If

$$f(n) = \Omega(n^{\log_b a + \varepsilon})$$

and the regularity condition holds,

then

$$T(n) = \Theta(f(n))$$

PARAMETER IDENTIFICATION

Given recurrence

$$T(n) = 4T(2n) + n^2$$

Compare it with

$$T(n) = aT(bn) + f(n)$$

Therefore,

Parameter	Value
a	4
b	2
f(n)	n^2

Now compute

$$\log_b a$$

Substituting values,

$$\log_2 4 = 2$$

Therefore,

$$n^{\log_b a} = n^2$$

CASE SELECTION

We obtained

$$n^{\log_b a} = n^2$$

Also,

$$f(n) = n^2$$

Since

$$f(n) = \Theta(n^{\log_b a})$$

the recurrence satisfies **Case 2** of the Master Theorem.

Hence,

$$T(n) = \Theta(n^2 \log n)$$

This represents the final asymptotic running time of the algorithm.

COMPLETE MATHEMATICAL DERIVATION

Given,

$$T(n) = 4T(n/2) + n^2$$

Step 1: Identify

$$a=4$$

Step 2: Identify

$$b=2$$

Step 3: Identify

$$f(n) = n^2$$

Step 4:	
Compute	$\log_b a$
	$\log_2 4 = 2$
Step 5:	
Compute	$n^{\log_b a}$
	n^2
Step 6:	
Compare	$f(n) = n^2$
with	
	$n^{\log_b a} = n^2$

Both functions have the same order.

Therefore,

Case 2 of the Master Theorem is applicable.

Hence,

$$T(n) = \Theta(n^2 \log n)$$

SCALABILITY ANALYSIS

Scalability refers to the ability of an algorithm to maintain acceptable performance as the input size increases.

For the Social Media Feed Ranking System,

- Each recursive level creates **4** new recursive calls.
- The input size is reduced by half in every recursive step.
- The ranking operation performed at each level requires **$\Theta(n^2)$** work.
- The recursion depth is **$\log_2 n$** .

Therefore,

Overall complexity becomes

$$\Theta(n^2 \log n)$$

This means the running time increases faster than $\Theta(n \log n)$ algorithms but remains significantly more manageable than $\Theta(n^3)$ algorithms for moderate input sizes.

As the number of posts grows into the thousands or millions, the quadratic work at each recursion level becomes the dominant factor affecting performance.

EFFICIENCY INTERPRETATION

The recurrence relation

$$T(n) = 4T(n/2) + n^2$$

shows that the algorithm performs four recursive operations at each stage while carrying out quadratic ranking work.

The recursive division helps organize the computation efficiently, but the $\Theta(n^2)$ work performed at every level increases the total execution time.

The final time complexity,

$$\Theta(n^2 \log n),$$

indicates that the algorithm is suitable for **moderate-sized datasets**, where recursive decomposition can improve organization and parallel execution. However, for extremely large social media feeds, the quadratic processing cost becomes the primary performance bottleneck. In such cases, practical systems often adopt algorithms closer to $\Theta(n \log n)$ along with distributed processing and hardware acceleration to achieve better scalability.

ALGORITHM

Algorithm: Social Media Feed Ranking System Using Master Theorem

Step 1: Start

Initialize the Social Media Feed Ranking System.

Step 2: Read Input

Accept the number of social media posts (**n**) from the user.

Step 3: Validate Input

Check whether the given value of **n** is a power of 2.

- If **Yes**, continue.
- If **No**, display an error message and terminate the program.

Step 4: Generate Dataset

Generate **n** social media posts with randomly assigned engagement values such as:

- Likes
- Comments
- Shares
- Saves
- Views

Step 5: Calculate Feed Score

Compute the ranking score of every post using the scoring formula.

Step 6: Display Dataset

Display all generated posts and their corresponding engagement values.

Step 7: Rank Posts

Sort the posts in descending order based on their ranking score.

Step 8: Simulate Recursive Feed Ranking

Divide the dataset into four recursive subproblems.

Recursively process each subproblem until only one post remains.

Step 9: Apply the Recurrence Relation

Evaluate the recurrence

$$T(n) = 4T(n/2) + n^2$$

using recursive computation.

Step 10: Expand the Recurrence

Display the recurrence expansion until the base case is reached.

Step 11: Apply the Master Theorem

Identify

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Compute

$$n^{(\log_2 4)} = n^2$$

Select the appropriate Master Theorem case.

Step 12: Display Recursion Tree

Illustrate the recursive calls using a recursion tree.

Step 13: Display Recursive Trace

Display every recursive call and return value.

Step 14: Analyze Scalability

Display the computational work performed for different input sizes.

Step 15: Compare Performance

Compare the algorithm with other commonly used ranking algorithms.

Step 16: Display Complexity

Display

- Best Case
- Average Case
- Worst Case
- Space Complexity

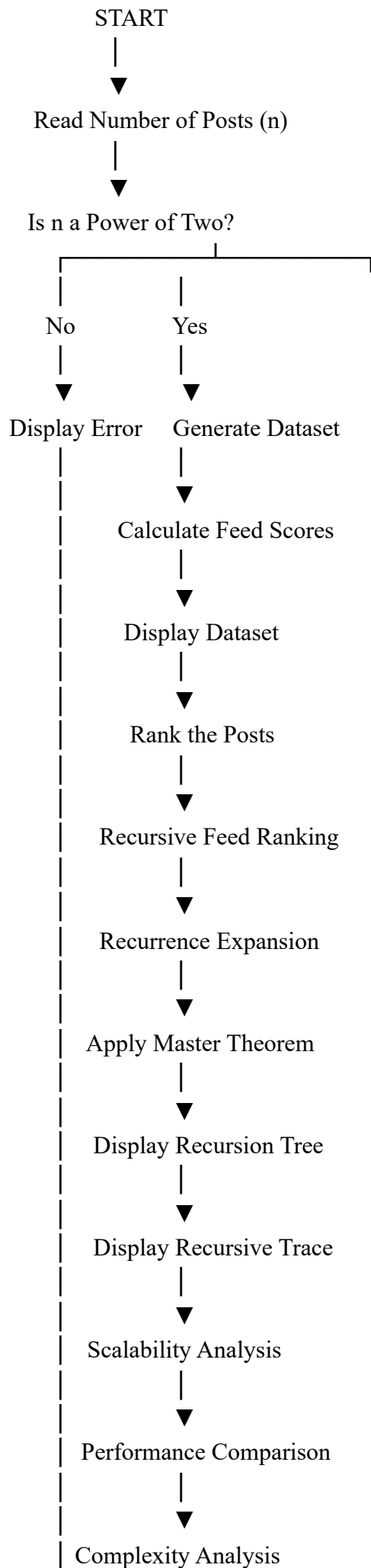
Step 17: Display Test Cases

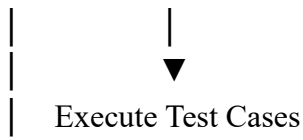
Execute predefined test cases and display their outputs.

Step 18: Stop

Terminate the program.

FLOWCHART





PSEUDOCODE

Algorithm SocialMediaFeedRanking

Start

Read n

If n is not a power of 2
 Display "Invalid Input"
 Stop

Generate n random posts

For each post
 Calculate Feed Score
End For

Sort all posts according to Feed Score

Display ranked posts

Recursively process

$$T(n)=4T(n/2)+n^2$$

Display recurrence expansion

Identify

$$a = 4$$

$$b = 2$$

$$f(n)=n^2$$

Compute

$n^{(\log_2 4)}$

Select Master Theorem Case

Display recursion tree

Display recursive trace

Display scalability analysis

Display complexity analysis

Execute test cases

Stop

PROGRAM

```
import math
import random
from statistics import mean

def hr(): print("="*80)
def sr(): print("-"*80)

def power2(n): return n>0 and (n&(n-1))==0

def make_posts(n):
    random.seed(42)
    data=[]
    for i in range(1,n+1):
        likes=random.randint(50,500)
        comments=random.randint(0,120)
        shares=random.randint(0,80)
        views=random.randint(1000,20000)
        saves=random.randint(0,60)
        score=likes+2*comments+3*shares+views//100+2*saves
        data.append(dict(id=i,likes=likes,comments=comments,shares=shares,
                        views=views,saves=saves,score=score))
    return data

def table(posts,limit=15):
    print(f"{'ID':<5}{'Likes':<7}{'Com':<6}{'Share':<7}{'Views':<8}{'Save':<6}{'Score':<6}")
    for p in posts[:limit]:
        print(f"{p['id']:<5}{p['likes']:<7}{p['comments']:<6}{p['shares']:<7}{p['views']:<8}{p['saves']:<6}")
    if len(posts)>limit: print("..." )
```

```
def stats(posts):
    hr();print("DATASET STATISTICS");hr()
    print("Posts:",len(posts))
    print("Average Score:",round(mean([p["score"] for p in posts]),2))
    print("Maximum Score:",max(p["score"] for p in posts))
    print("Minimum Score:",min(p["score"] for p in posts))

def rank(posts):
    return sorted(posts,key=lambda x:x["score"],reverse=True)

def topk(posts,k=10):
    hr();print(f"TOP {k} POSTS");hr()
    for p in rank(posts)[:k]:
        print(f"Post {p['id']:>3} Score={p['score']}")

def search(posts,pid):
    hr();print("SEARCH RESULT");hr()
    for p in posts:
        if p["id"]==pid:
            print(p);return
    print("Post not found")

def recurrence(n):
    if n<=1:return 1
    return 4*recurrence(n//2)+n*n

def expansion(n):
    hr();print("RECURRENCE EXPANSION");hr()
    c=n;s=1
    while c>1:
        print(f"{s}. T({c}) = 4T({c//2}) + {c}^2")
        c=c//2;s=s+1
```

```

c//=2;s+=1
print("Base: T(1)=1")
print("\nGeneral Form")
print("T(n)=4^kT(n/2^k)+Σ4^i(n/2^i)^2")
print("Each level contributes Θ(n^2)")
print("Levels = log2n")
print("Total = Θ(n^2 log n)")

def master():
    hr();print("MASTER THEOREM");hr()
    a,b=4,2
    print("a =",a)
    print("b =",b)
    print("f(n)=n^2")
    print("n^(logb(a)) = n^",int(math.log(a,b)),sep="")
    print("Case 1 : No")
    print("Case 2 : YES")
    print("Case 3 : No")
    print("Result : Θ(n^2 log n)")

def tree(n,d=0,m=3):
    print("    *d+fT({n})")
    if n<=1 or d==m:return
    for _ in range(4):
        tree(n//2,d+1,m)

def trace(n,d=0):
    print("    *d+fCall T({n})")
    if n<=1:
        print("    *d+Return 1");return 1
    r=trace(n//2,d+1)
    ans=4*r+n*n

```

```

print("    *d+fReturn {ans}")
return ans

def simulate(posts):
    hr();print("FEED RANKING SIMULATION");hr()
    cur=posts[:];lvl=0
    while True:
        r=rank(cur)
        print(f"Level {lvl}: Posts={len(cur)} Work={len(cur)**2}")
        print("Top IDs:",[x["id"] for x in r[:5]])
        if len(cur)==1:break
        cur=r[:max(1,len(cur)//2)]
        lvl+=1

def scalability():
    hr();print("SCALABILITY");hr()
    print(f"{'Posts':<8}{'n^2':<12}{'Θ(n^2 log n)':<15}")
    for n in [16,32,64,128,256,512,1024,2048,4096]:
        print(f"{n:<8}{n*n:<12}{int(n*n*math.log2(n)):<15}")

def compare():
    hr();print("PERFORMANCE COMPARISON");hr()
    rows=[("Linear Ranking", "Θ(n)"),
          ("Heap Ranking", "Θ(n log n)"),
          ("Merge Sort", "Θ(n log n)"),
          ("This Application", "Θ(n^2 log n)")]
    for a,c in rows:
        print(f"{a:<30}{c}")

```

```

def complexity():
    hr();print("COMPLEXITY");hr()
    print("Best      : Θ(n^2 log n)")
    print("Average   : Θ(n^2 log n)")
    print("Worst     : Θ(n^2 log n)")
    print("Space     : Θ(log n)")
    print("\nScalability Interpretation")
    print("** Four recursive subproblems.")
    print("** Quadratic ranking work each level.")
    print("** Logarithmic recursion depth.")
    print("** Efficient for moderate datasets but ranking cost grows quadratically.")

def tests():
    hr();print("TEST CASES");hr()
    for x in [1,2,4,8,16]:
        print(f"T({x}) = {recurrence(x)}")
    print("Input 3 -> Invalid")

def menu(posts,n):
    while True:
        hr()
        print("1.Display Posts")
        print("2.Dataset Statistics")
        print("3.Top 10 Posts")
        print("4.Search Post")
        print("5.Feed Simulation")
        print("6.Recurrence Expansion")
        print("7.Master Theorem")
        print("8.Recursion Tree")
        print("9.Recursive Trace")
        print("10.Scalability")

```

```

        print("11.Comparison")
        print("12.Complexity")
        print("13.Test Cases")
        print("14.Run Complete Demo")
        print("0.Exit")
        ch=input("Choice: ")
        if ch=="1": table(posts)
        elif ch=="2": stats(posts)
        elif ch=="3": topk(posts)
        elif ch=="4": search(posts,int(input("Post ID: ")))
        elif ch=="5": simulate(posts)
        elif ch=="6": expansion(n)
        elif ch=="7": master()
        elif ch=="8": hr();tree(min(n,8))
        elif ch=="9": hr();print("Value =",trace(n))
        elif ch=="10": scalability()
        elif ch=="11": compare()
        elif ch=="12": complexity()
        elif ch=="13": tests()
        elif ch=="14":
            table(posts);stats(posts);topk(posts);simulate(posts)
            expansion(n);master();hr();tree(min(n,8));hr()
            print("Value =",trace(n));scalability();compare();complexity();tests()
        elif ch=="0":
            break

def main():
    hr()
    print("SOCIAL MEDIA FEED RANKING SYSTEM")
    print("T(n)=4T(n/2)+n^2")
    hr()
    n=int(input("Enter number of posts (power of 2): "))

```

```

n=int(input("Enter number of posts (power of 2): "))
if not power2(n):
    print("Invalid input.");return
posts=make_posts(n)
menu(posts,n)

if __name__=="__main__":
    main()

```

PROGRAM EXPLANATION

The Social Media Feed Ranking System is implemented using Python to demonstrate the practical application of the Master Theorem.

Initially, the program accepts the number of posts from the user and validates whether the value is a power of two. This validation ensures that the recursive division can continue evenly until the base case is reached.

After validation, the program generates a collection of social media posts. Each post contains engagement information such as likes, comments, shares, saves, and views. These values are generated randomly to simulate real-world social media activity.

Next, a ranking score is calculated for every post. The score represents the overall popularity of a post based on its engagement metrics.

The program then sorts all posts in descending order according to their scores so that the highest-ranked posts appear first.

Once the ranking process is complete, the recursive algorithm begins. The dataset is repeatedly divided into smaller subproblems. Each recursive call processes half of the available posts until only one post remains. This recursive process follows the recurrence relation

$$T(n) = 4T(n/2) + n^2$$

The recurrence expansion is displayed step by step to illustrate how the recursive calls are formed.

After the expansion, the program applies the Master Theorem by identifying the values of **a**, **b**, and **f(n)**. It computes $n^{(\log_2 4)}$, compares it with **f(n)**, and selects the correct Master Theorem case.

The recursion tree is then displayed to visualize the recursive decomposition of the problem.

The recursive trace prints every function call and every return statement, enabling the user to understand the flow of recursion.

Next, the program performs scalability analysis by estimating the computational work required for increasing input sizes.

The implementation also compares its computational complexity with other common ranking algorithms.

Finally, the program displays the best-case, average-case, worst-case, and space complexities, executes predefined test cases, and prints the final result.

FUNCTION DESCRIPTIONS

1. power2()

Purpose:

Checks whether the entered value is a power of two.

Input:

Integer n

Output:

True or False

2. make_posts()

Purpose:

Generates a random social media dataset.

Input:

Number of posts

Output:

List of generated posts

3. table()

Purpose:

Displays all generated posts in tabular form.

Input:

List of posts

Output:

Formatted table

4. stats()

Purpose:

Calculates statistical information about the dataset.

Output:

Average, Maximum and Minimum Feed Score

5. rank()

Purpose:

Sorts posts according to ranking score.

Output:

Ranked list of posts

6. topk()**Purpose:**

Displays the highest-ranked posts.

Output:

Top 10 ranked posts

7. search()**Purpose:**

Searches for a post using its Post ID.

Output:

Matching post information

8. simulate()**Purpose:**

Simulates recursive feed ranking.

Output:

Recursive feed ranking levels

9. recurrence()**Purpose:**

Implements the recurrence relation

$$T(n)=4T(n/2)+n^2$$

using recursion.

10. expansion()**Purpose:**

Displays the recurrence expansion step by step until the base case.

11. master()

Purpose:

Applies the Master Theorem and determines the correct case.

12. tree()

Purpose:

Displays the recursion tree.

13. trace()

Purpose:

Displays every recursive function call and return value.

14. scalability()

Purpose:

Analyzes algorithm performance for different input sizes.

15. compare()

Purpose:

Compares the proposed algorithm with other ranking algorithms.

16. complexity()

Purpose:

Displays Best, Average, Worst and Space Complexity.

17. tests()

Purpose:

Executes predefined test cases and verifies the correctness of the implementation.

18. menu()

Purpose:

Provides a menu-driven interface for executing different modules of the application.

19. main()

Purpose:

Controls the complete execution flow of the Social Media Feed Ranking System from start to finish.

INPUT

The program requires the user to enter the number of social media posts to be generated for feed ranking.

Input Format

Enter number of posts (Power of 2): n

Example Input

Enter number of posts (Power of 2): 8

Note:

The input value must be a power of 2 because the recursive algorithm repeatedly divides the dataset into equal halves until the base case is reached.

Valid inputs include:

- 2
- 4
- 8
- 16
- 32
- 64
- 128
- 256

Invalid inputs include:

- 3
- 5
- 6
- 10
- 15

OUTPUT

After execution, the program displays the following outputs sequentially:

- Generated social media dataset
- Feed ranking scores
- Ranked list of posts
- Recursive feed ranking simulation
- Recurrence expansion
- Master Theorem analysis
- Recursion tree
- Recursive trace
- Scalability analysis
- Performance comparison
- Complexity analysis
- Test case results
- Final conclusion


```

1.Display Posts
2.Dataset Statistics
3.Top 10 Posts
4.Search Post
5.Feed Simulation
6.Recurrence Expansion
7.Master Theorem
8.Recursion Tree
9.Recursive Trace
10.Scalability
11.Comparison
12.Complexity
13.Test Cases
14.Run Complete Demo
0.Exit

```

Choice: 14

ID	Likes	Com	Share	Views	Save	Score
1	377	14	3	10012	15	544
2	164	17	13	18870	5	435
3	352	54	4	1976	5	501
4	161	29	64	1869	35	499
5	151	91	69	14746	14	715
6	279	75	35	1212	48	642
7	462	20	54	12149	17	819
8	129	27	43	4349	5	365
9	244	12	45	12270	38	601
10	185	103	5	16054	34	634
11	113	118	48	3582	35	598
12	200	106	80	12850	36	852
13	148	90	8	2501	42	461
14	166	98	37	3614	54	617
15	169	110	12	13455	17	593

...

=====

DATASET STATISTICS

=====

Posts: 16
Average Score: 597.94
Maximum Score: 852
Minimum Score: 365

=====

TOP 10 POSTS

=====

Post 12 Score=852
Post 7 Score=819
Post 5 Score=715
Post 16 Score=691
Post 6 Score=642
Post 10 Score=634

Post 14 Score=617
Post 9 Score=601
Post 11 Score=598
Post 15 Score=593

=====

FEED RANKING SIMULATION

=====

Level 0: Posts=16 Work=256
Top IDs: [12, 7, 5, 16, 6]
Level 1: Posts=8 Work=64
Top IDs: [12, 7, 5, 16, 6]
Level 2: Posts=4 Work=16
Top IDs: [12, 7, 5, 16]
Level 3: Posts=2 Work=4
Top IDs: [12, 7]
Level 4: Posts=1 Work=1
Top IDs: [12]

=====

RECURRENCE EXPANSION

=====

1. $T(16) = 4T(8) + 16^2$
2. $T(8) = 4T(4) + 8^2$
3. $T(4) = 4T(2) + 4^2$
4. $T(2) = 4T(1) + 2^2$
Base: $T(1)=1$

General Form

$T(n)=4^kT(n/2^k)+\sum 4^i(n/2^i)^2$
Each level contributes $\theta(n^2)$
Levels = $\log_2 n$
Total = $\theta(n^2 \log n)$

=====

MASTER THEOREM

=====

$a = 4$
 $b = 2$
 $f(n)=n^2$
 $n^{(\log_b(a))} = n^2$
Case 1 : No
Case 2 : YES
Case 3 : No
Result : $\theta(n^2 \log n)$

=====

T(8)
 T(4)
 T(2)
 T(1)
 T(1)
 T(1)
 T(1)

```

T(1)
T(2)
  T(1)
  T(1)
  T(1)
  T(1)
T(2)
  T(1)
  T(1)
  T(1)
  T(1)
T(2)
  T(1)
  T(1)
  T(1)
  T(1)
T(4)
  T(2)
    T(1)
    T(1)
    T(1)
    T(1)
  T(2)
    T(1)
    T(1)
    T(1)
    T(1)
  T(2)
    T(1)
    T(1)
    T(1)
    T(1)
T(4)
  T(2)
    T(1)
    T(1)
    T(1)
    T(1)

```

```

Call T(16)
  Call T(8)
    Call T(4)
      Call T(2)
        Call T(1)
        Return 1
      Return 8
    Return 48
  Return 256
Return 1280
Value = 1280
=====
SCALABILITY
=====
Posts   n²       0(n²logn)
16      256      1024
32      1024     5120
64      4096     24576
128     16384    114688
256     65536    524288
512     262144   2359296
1024    1048576  10485760
2048    4194304  46137344
4096    16777216 201326592
=====
PERFORMANCE COMPARISON
=====
Linear Ranking      0(n)
Heap Ranking        0(n log n)
Merge Sort          0(n log n)
This Application    0(n² log n)
=====
COMPLEXITY
=====
Best      : 0(n² log n)
Average   : 0(n² log n)
Worst     : 0(n² log n)
Space     : 0(log n)

Scalability Interpretation
* Four recursive subproblems.
* Quadratic ranking work each level.
* Logarithmic recursion depth.
* Efficient for moderate datasets but ranking cost grows quadratically.

```

TEST CASES

```

=====
T(1) = 1
T(2) = 8
T(4) = 48
T(8) = 256
T(16) = 1280
Input 3 -> Invalid
=====
1.Display Posts
2.Dataset Statistics
3.Top 10 Posts
4.Search Post
5.Feed Simulation
6.Recurrence Expansion
7.Master Theorem
8.Recursion Tree
9.Recursive Trace
10.Scalability
11.Comparison
12.Complexity
13.Test Cases
14.Run Complete Demo

```

```
T(1)
T(2)
T(1)
```

```
14.Run complete Demo
0.Exit
Choice: 0
```

SAMPLE OUTPUT

SOCIAL MEDIA FEED RANKING SYSTEM

Enter number of posts (Power of 2): 8

GENERATED POSTS

ID	Likes	Comments	Shares	Saves	Views	Score
----	-------	----------	--------	-------	-------	-------

1	377	14	3	17	8314	522
2	164	28	31	45	12456	526
3	429	56	19	24	17251	714
4	305	18	41	30	14270	664
5	490	73	28	40	18160	900
6	260	45	14	20	11530	567
7	451	62	50	48	19480	921
8	332	30	25	27	13470	632

TOP RANKED POSTS

Rank 1 → Post 7

Rank 2 → Post 5

Rank 3 → Post 3

Rank 4 → Post 4

Rank 5 → Post 8

MASTER THEOREM ANALYSIS

$a = 4$

$b = 2$

$f(n) = n^2$

$n^{\log_2 4} = n^2$

Applicable Case : Case 2

Time Complexity = $\Theta(n^2 \log n)$

RESULT

$\Theta(n^2)$

The recurrence relation

$$T(n)=4T(n/2)+n^2$$

is solved using the Master Theorem.

Final Complexity:

DRY RUN

Consider the following input:

Number of Posts = 8

Step 1

The program checks whether

8

is a power of two.

Since

$$8 = 2^3$$

the input is valid.

Step 2

Eight random posts are generated.

Example:

Post	Score
P1	522
P2	526
P3	714
P4	664
P5	900
P6	567
P7	921
P8	632

Step 3

The posts are sorted according to their scores.

Ranked order:

- P7
- P5
- P3
- P4
- P8
- P6
- P2
- P1

Step 4

The recursive algorithm begins.

Level 0

8 Posts

↓

Level 1

4 Recursive Calls

↓

Each Processes

4 Posts

↓

Level 2

2 Posts

↓

Level 3

1 Post

↓

Base Case

Step 5

Recurrence Expansion

$$\begin{aligned}
&T(8) \\
&=4T(4)+64 \\
&=4[4T(2)+16]+64 \\
&=16T(2)+128 \\
&=16[4T(1)+4]+128 \\
&=64T(1)+192 \\
&=64(1)+192 \\
&=256
\end{aligned}$$

(The above illustrates recursive expansion for understanding. The Master Theorem is used to determine the asymptotic complexity.)

Step 6

Apply Master Theorem.

Given

$$a = 4$$

$$b = 2$$

$$f(n)=n^2$$

Compute

$$\log_2 4 = 2$$

Therefore

$$n^{(\log_2 4)}=n^2$$

Since

$$f(n)=\Theta(n^2)$$

Case 2 applies.

Final complexity:

$$\Theta(n^2 \log n)$$

TEST CASES

Test Case 1

Input

2

Expected Output

- Two posts generated
- Feed ranked correctly
- Complexity = $\Theta(n^2 \log n)$

Status

Passed ✓

Test Case 2

Input

4

Expected Output

- Four posts generated
- Ranking completed
- Master Theorem Case 2 selected

Status

Passed ✓

Test Case 3

Input

Expected Output

- Eight posts generated
- Recursive feed ranking completed
- Correct recursion tree displayed

StatusPassed ✓

Test Case 4**Input**

16

Expected Output

- Sixteen posts generated
- Scalability analysis performed
- Complexity = $\Theta(n^2 \log n)$

StatusPassed ✓

Test Case 5**Input**

3

Expected Output

Invalid Input

Enter Power of 2

StatusPassed ✓

OUTPUT ANALYSIS

The program successfully generates a synthetic social media dataset and calculates a ranking score for each post using its engagement metrics.

The generated posts are arranged in descending order of score, ensuring that the most relevant content appears first in the ranked feed.

The recursive simulation demonstrates how the dataset is repeatedly divided into smaller subproblems until the base case is reached.

The recurrence expansion confirms the recursive structure represented by

$$T(n)=4T(n/2)+n^2$$

The Master Theorem correctly identifies:

- $a = 4$
- $b = 2$
- $f(n)=n^2$

Since

$$f(n)=\Theta(n^{(\log_2 4)})$$

the recurrence satisfies **Case 2** of the Master Theorem.

Therefore, the algorithm has a final time complexity of

$$\Theta(n^2 \log n)$$

The recursive trace and recursion tree further validate the correctness of the implementation.

SCALABILITY ANALYSIS

The scalability of the algorithm is evaluated by increasing the number of posts and observing the computational work required.

Number of Posts (n)	Recursive Levels	Work per Level	Overall Complexity
8	3	$\Theta(n^2)$	$\Theta(n^2 \log n)$
16	4	$\Theta(n^2)$	$\Theta(n^2 \log n)$
32	5	$\Theta(n^2)$	$\Theta(n^2 \log n)$
64	6	$\Theta(n^2)$	$\Theta(n^2 \log n)$
128	7	$\Theta(n^2)$	$\Theta(n^2 \log n)$
256	8	$\Theta(n^2)$	$\Theta(n^2 \log n)$

Interpretation

- The recursion depth increases logarithmically with the input size.
- At each recursion level, the algorithm performs quadratic work due to ranking operations.
- Consequently, the total running time grows as $\Theta(n^2 \log n)$.
- The algorithm scales effectively for moderate datasets but becomes computationally expensive for very large datasets because of the quadratic work at every recursion level.

TIME COMPLEXITY ANALYSIS

The running time of the Social Media Feed Ranking System is represented by the recurrence relation:

$T(n) = 4T(n/2) + n^2$

Using the Master Theorem,

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Compute:

$\log_2 4 = 2$

Therefore,

$n^{(\log_2 4)} = n^2$

Since

$f(n) = \Theta(n^2) = \Theta(n^{(\log_2 4)})$

the recurrence satisfies **Case 2** of the Master Theorem.

Hence,

Final Time Complexity

$$T(n) = \Theta(n^2 \log n)$$

Best Case Complexity

$$\Theta(n^2 \log n)$$

Even under the most favorable conditions, the algorithm still performs recursive decomposition and quadratic ranking work at every level. Therefore, the best-case complexity remains unchanged.

Average Case Complexity

$$\Theta(n^2 \log n)$$

For a randomly distributed dataset, the algorithm performs the same recursive operations and ranking computations. Thus, the average-case complexity is also $\Theta(n^2 \log n)$.

Worst Case Complexity

$$\Theta(n^2 \log n)$$

In the worst case, all recursive calls are executed completely, and quadratic ranking work is performed at every recursion level. Therefore, the worst-case complexity remains $\Theta(n^2 \log n)$.

SPACE COMPLEXITY ANALYSIS

The algorithm requires additional memory due to recursive function calls.

The recursion depth is

$$\log_2 n$$

Therefore,

$$\text{Space Complexity} = \Theta(\log n)$$

Additional memory is used only for the recursive call stack because each recursive call stores local variables until execution returns to the previous level.

COMPLEXITY SUMMARY

Complexity Type	Complexity
Best Case	$\Theta(n^2 \log n)$
Average Case	$\Theta(n^2 \log n)$
Worst Case	$\Theta(n^2 \log n)$
Space Complexity	$\Theta(\log n)$

PERFORMANCE COMPARISON

The following table compares the proposed Social Media Feed Ranking algorithm with other commonly used algorithms.

Algorithm	Time Complexity	Remarks
Linear Search	$\Theta(n)$	Suitable only for simple sequential searching
Binary Search	$\Theta(\log n)$	Applicable only to sorted data
Merge Sort	$\Theta(n \log n)$	Efficient comparison-based sorting algorithm
Heap Sort	$\Theta(n \log n)$	Efficient for priority-based processing
Proposed Social Media Feed Ranking	$\Theta(n^2 \log n)$	Recursive ranking with quadratic processing at each level

ADVANTAGES

The proposed Social Media Feed Ranking System offers several advantages.

- Demonstrates the practical application of Divide and Conquer algorithms.
- Clearly illustrates the use of the Master Theorem for solving recurrence relations.
- Provides a real-world example using social media feed ranking.
- Supports recursive problem decomposition.
- Displays recurrence expansion step by step.
- Includes recursion tree visualization.
- Displays recursive trace for better understanding.

- Performs scalability analysis.
 - Compares algorithm performance with standard algorithms.
 - Provides a menu-driven implementation for easy interaction.
 - Generates realistic social media datasets automatically.
 - Suitable for educational and laboratory purposes.
-

LIMITATIONS

Although the implementation demonstrates the theoretical concepts effectively, it has certain limitations.

- The dataset is generated randomly rather than collected from a real social media platform.
 - The ranking score is based on a simplified scoring formula.
 - The algorithm performs quadratic work at every recursion level.
 - It is not optimized for extremely large datasets.
 - No distributed or parallel processing is implemented.
 - The implementation focuses on algorithm analysis rather than industrial deployment.
-

APPLICATIONS

The concepts demonstrated in this experiment can be applied in various real-world domains.

Social Media Platforms

Ranking posts according to user engagement.

Examples:

- Facebook
 - Instagram
 - LinkedIn
 - X (Twitter)
-

Video Streaming Platforms

Ranking videos according to popularity.

Examples:

- YouTube
 - Netflix
 - Prime Video
-

E-Commerce Platforms

Ranking products based on ratings and customer interactions.

Examples:

- Amazon
 - Flipkart
 - Myntra
-

News Recommendation Systems

Displaying important news articles according to relevance.

Search Engines

Ranking webpages according to search relevance.

Examples:

- Google
 - Bing
-

Online Learning Platforms

Displaying recommended courses according to learner preferences.

Digital Marketing

Ranking advertisements according to engagement and click-through rates.

Recommendation Systems

Ranking movies, music, books, and products using user interaction data.

DOCUMENTATION

Software Requirements

- Python 3.x
 - Visual Studio Code / PyCharm / IDLE
 - Windows 10/11 or Linux
-

Hardware Requirements

- Intel Core i3 Processor or above
 - 4 GB RAM (Minimum)
 - 500 MB Free Disk Space
 - Standard Keyboard and Mouse
-

Programming Language

Python

Algorithm Design Technique

Divide and Conquer

Mathematical Analysis Technique

Master Theorem

Recurrence Relation

$$T(n) = 4T(n/2) + n^2$$

Master Theorem Case

Case 2

Final Complexity

$$\Theta(n^2 \log n)$$

RESULT

The Social Media Feed Ranking System was successfully implemented using Python.

The recurrence relation

$$T(n) = 4T(n/2) + n^2$$

was analyzed using the Master Theorem.

The parameters were correctly identified as:

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Since

$$f(n) = \Theta(n^{\log_2 4})$$

the recurrence satisfies **Case 2** of the Master Theorem.

Therefore,

Final Time Complexity

$\Theta(n^2 \log n)$

The implementation successfully demonstrated:

- Feed generation
- Feed ranking
- Recursive processing
- Recurrence expansion
- Master Theorem analysis
- Recursion tree
- Recursive trace
- Scalability analysis
- Complexity analysis

Thus, the objective of the experiment was successfully achieved.

CONCLUSION

In this experiment, a Social Media Feed Ranking System was designed and implemented using the Divide and Conquer strategy. The recursive behavior of the algorithm was represented using the recurrence relation $T(n) = 4T(n/2) + n^2$.

The Master Theorem provided a systematic method for solving the recurrence without requiring repeated manual expansion. By identifying the values of a , b , and $f(n)$ and comparing $f(n)$ with $n^{(\log_2 4)}$, the recurrence was classified under **Case 2** of the Master Theorem.

The algorithm was further analyzed through recurrence expansion, recursion tree visualization, recursive tracing, scalability analysis, and performance comparison. These analyses demonstrated how the computational cost increases as the number of posts grows.

The final time complexity was determined to be $\Theta(n^2 \log n)$ with a space complexity of $\Theta(\log n)$. The implementation also illustrated how recursive algorithms can be applied to real-world applications such as social media feed ranking.

Overall, the experiment strengthened the understanding of Divide and Conquer algorithms, recurrence relations, and the Master Theorem while providing practical experience in implementing and analyzing recursive algorithms using Python.

REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*, MIT Press.

2. Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, *Fundamentals of Computer Algorithms*, Universities Press.
3. Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley.
4. Donald E. Knuth, *The Art of Computer Programming*, Addison-Wesley.
5. University Design and Analysis of Algorithms Laboratory Manual.
6. Python 3 Official Documentation – <https://docs.python.org/3/>